



Hidden in Plain Sight:

A Playful Look at Everyday Security Risks

Anthea (an-THAY-ah)
Gonzalez-Santiago

cackalackycon 2026



What I am going to yap about today?

- Me (briefly)
- Our Environments (also briefly)
- Issues → Incidents
- Why our Issues Linger
- “Security” Loteria!
- Pattern Analysis
- Practical Takeaway Checklist
- Q&A



Who is *she*? 🙌

- Anthea (an-THAY-ah) Gonzalez-Santiago or theagon (thea-ah-gone)
- Security Engineer
- Oak City Locksport Volunteer
- WiCyS Latina Board Member
 - Se habla Español 🇲🇽
- Reading, Coloring, Gaming, 🐞, 🌀101



in: <https://www.linkedin.com/in/antheaglz/>

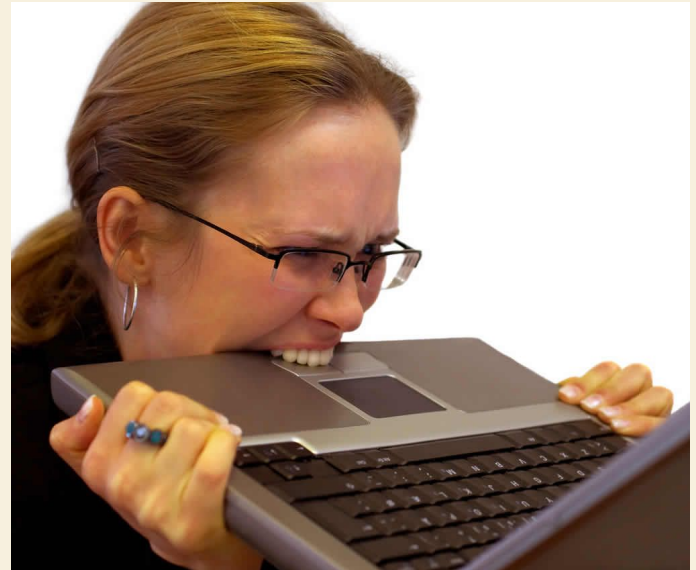
Lets think about our environments...

- Just **think**
- Processes? Connections? Configurations?
- Also think “non-technical”!
- ...What stuff makes your tummy hurt?
- Now that we have a few things in mind...



Issues → Incident

- We have seen how these become REALLY bad, really fast.
 - Equifax
- Raising the alarm, again and again
- Issue + Timing/Circumstance = 🚨
- Not fancy or complex, simple things



Why These Issues Linger

Human Patterns

- “Later” Mentality
 - Acknowledge it, fix it later.
- Optimism Bias
 - “It won’t happen to us.”
- Convenience > Security
 - Quick fixes, long-term risks.



Why These Issues Linger

Organizational Patterns

- Unclear Ownership
 - Who's responsible?
- Shifting Priorities
 - Security gets sidelined.
- Resource Limitations
 - Staff stretched thin.



Why These Issues Linger

Technical Patterns

- Complexity
 - Too much to track.
- Legacy Systems
 - Old tech, new risks.
- Invisible Dependencies
 - Unknown risks.



“Security” Loteria!

- I will “draw” a card aka advance the slides
- Read the verse
- Give the audience a chance to guess
- I will reveal the card, mark it off if you have it!
- Unpack the meaning of the card
- Winner: Must have all of their images marked off, Shout out “Loteria”!

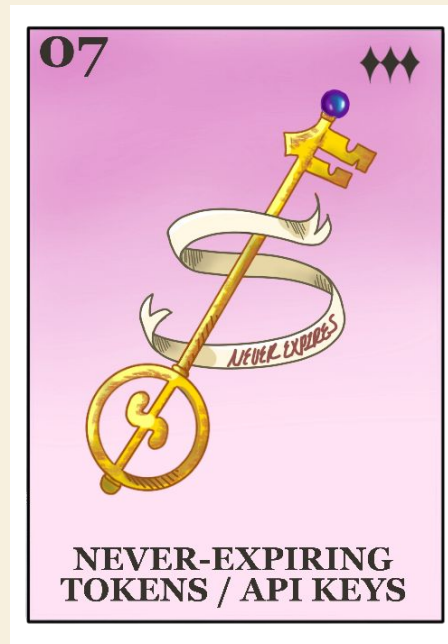


Never-expiring Tokens / API Keys

“Created once, trusted always.”

A secret that outlives its purpose.”

- Tokens/keys issued with no expiration stay valid
- Leaked keys provide long-term, silent access
- No rotation → no detection of compromise
- Old keys accumulate and complicate audits

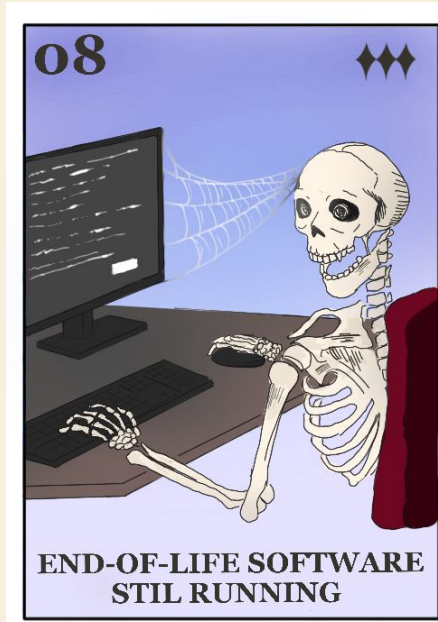


End-of-life Software Still Running

“Support has ended, but the system remains.

Old code persists, carrying new risks.”

- Unsupported software receives no security patches
- Known CVEs remain permanently exploitable
- Blocks upgrades and modern tooling
- Creates unpatchable entry points



Insecure Default Configurations

“Left as-is because it worked.”

Convenience today, compromise tomorrow.”

- Systems deployed with vendor defaults
- Defaults are predictable and widely scanned for
- Creates persistent misconfigurations
- Harder to fix later without downtime or migration



Orphaned Resources

“Still running alone, forgotten by people but not by attackers, a ghost with compute.”

- Cloud resources with no owner or maintenance
- Unpatched/unmonitored systems become entries
- Public resources leak data silently
- Wasted spend on unused infrastructure

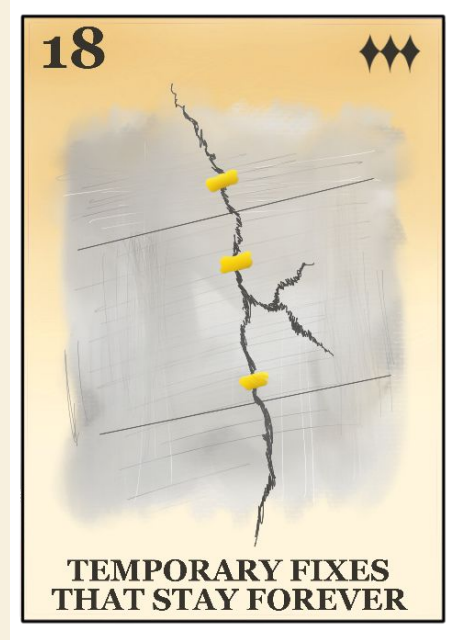


Temporary Fixes That Stay Forever

“Made for a moment, kept far too long.”

A quick fix settling in like it belongs.”

- Quick workaround becomes permanent and fragile
- Masks deeper failures that worsen over time
- Undocumented behavior escalates incidents
- Creates long-term attack surface

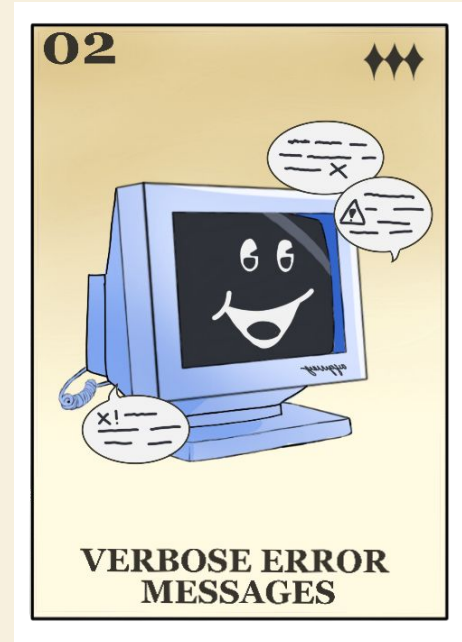


Verbose Error Messages

“A tiny failure, a generous confession.

Secrets spill out to anyone who listens.”

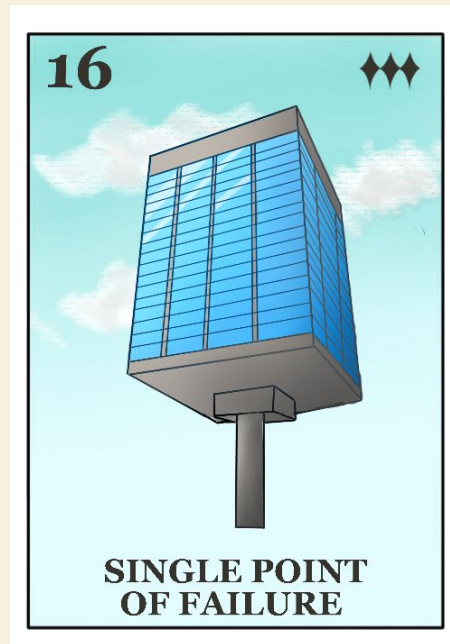
- Errors reveal internal details
- Helps attackers map architecture
- Leaks sensitive data through responses or logs
- Enables precise, targeted payloads



Single Point of Failure

*“One system, one person,
holding everything together until they don’t.”*

- One component failure breaks an entire workflow
- Outages cascade from a single failure
- Maintenance can causes downtime
- Attackers target the choke point

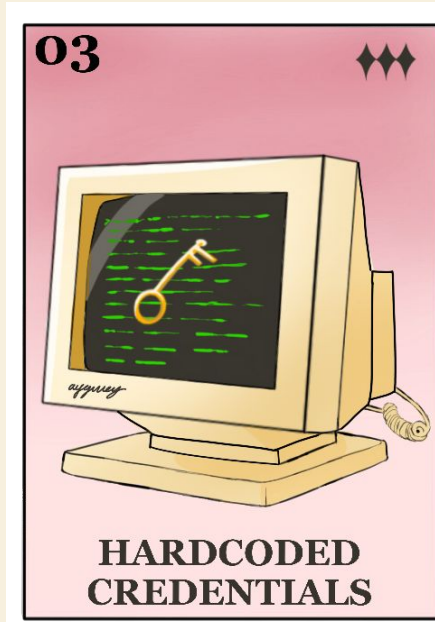


Hardcoded Credentials

“Passwords buried in code, meant to be temporary.

They linger like long after deployment.”

- Secrets embedded in code, binaries, scripts, configs
- Secrets leak through repos, logs, screenshots
- Rotation requires code changes and redeploys
- Attackers scan repos for creds at scale



Untested DR Plan / Untested Backups

“Written down, but never tried.”

A promise of safety that no one has verified.

- Backups/DR plans exist but are never validated
- Backups fail silently until needed
- Recovery becomes slow or impossible
- Runbooks outdated or incomplete



Pattern Analysis

- They start small.
- They hide in plain sight.
- They accumulate quietly.
- They become incidents only when timing is unlucky.
- They're preventable with small, consistent habits.



Practical Takeaway Checklist

- ☐ Identify what's lingering
- ☐ Validate what still needs to exist
- ☐ Rotate keys, tokens, credentials
- ☐ Patch images, dependencies, configs
- ☐ Limit permissions, sessions, resources
- ☐ Review ownership, defaults, exceptions
- ☐ Test backups, DR, assumptions





Thank you for attention and participation!

Q/A

